TWITTER / X — SYSTEM DESIGN & INTERVIEW REFERENCE V1

The fan-out problem, and everything that follows from it

capacity · CAP per capability · fan-out · Snowflake ids · timeline & ranking · graph · search · trends · failures · **depth marked** junior → staff

1 · REQUIREMENTS, SCOPE AND THE ONE PROBLEM THAT DEFINES THIS DESIGN

start here — what to establish first, and why this system is interesting at all

Functional scope — say what is out, too

In	Post a tweet (text, media) · follow and unfollow · **home timeline** · user timeline · search · trends · likes, replies and retweets · notifications
Out	Direct messages, spaces, advertising and monetisation, moderation policy, recommendation model internals — *named explicitly, so the design has an edge*

Non-functional targets, with percentiles

Home timeline read	**p99 < 200 ms**
Post tweet	p99 < 500 ms to acknowledge
Tweet visible to followers	Seconds, not milliseconds — this is a **deliberate** relaxation
Search freshness	Seconds
Availability — read	99.99%
Availability — write	99.9% is the honest figure; a failed post is recoverable, a lost one is not

Availability arithmetic, stated properly. Serial dependencies **multiply**: a read path touching five services at 99.9% each yields roughly 99.5% composite — about 44 hours a year. Quoting 99.99% for a long synchronous chain is arithmetic, not ambition, which is why so much of this design pushes work off the read path.

The defining characteristic

This is a read-dominated system with an extreme fan-out asymmetry. Reads outnumber writes by roughly two orders of magnitude — and one write can require millions of downstream writes, or none, depending entirely on *who* posted it. Nearly every decision on this sheet exists to manage that asymmetry.

Why the home timeline is the hard part

A home timeline is a merged, ranked view of tweets from everyone you follow. Computing it naively at read time means querying hundreds of users' recent tweets, merging and ranking them — **per request, at hundreds of thousands of requests per second**. That is not achievable, so the timeline is precomputed. Precomputing it is what creates the celebrity problem.

The follower distribution is the whole story

User	Followers	Fan-out cost per tweet
Median user	~ hundreds	Trivial
Popular account	~ 100 K	Noticeable
Celebrity	10 M – 100 M+	**Catastrophic if handled the same way**

This is a power-law distribution, not a normal one. Designing for the average user produces a system that collapses on the tail — and the tail is precisely the traffic that matters most commercially.

How to work through it in an interview

1. **Clarify and scope** — features, scale, and what is out
2. **Estimate** — derive QPS and storage, stating assumptions aloud
3. **Define the API** — it forces the data model into the open
4. **Data model and access patterns**, then the stores that fit them
5. **High-level architecture** — happy path only
6. **Deep-dive the fan-out**. This is what the question is actually about
7. **Bottlenecks, failure modes, degradation**
8. **Trade-offs you would revisit**, and on what signal

Depth expected by level. JUNIOR the components and the read/write split. MID fan-out on write versus read, and caching. STAFF the hybrid boundary and how it is chosen, ranking, ordering guarantees, deletion and edit propagation, cost, and what breaks first. Markers on this sheet indicate where each depth begins.

The most common interview mistake is going straight to boxes and arrows without establishing the follower distribution. Without that, the celebrity problem never surfaces — and it is the entire reason this question is asked.

2 · CAPACITY ESTIMATION

assumptions, then arithmetic

State these as assumptions, not facts. Public figures vary by source and by year. What matters in an interview is that the arithmetic is right and the reasoning is visible — not that the inputs match a press release.

```
// assumptions
monthly active users            500,000,000
daily active users (~50%)       250,000,000
tweets per DAU per day                   ~2
timeline opens per DAU per day          ~20

// writes
500M tweets/day ÷ 86,400   ≈  5,800 tweets/sec
peak (×3)                  ≈ 17,000 tweets/sec

// reads
5B timeline views/day ÷ 86,400 ≈ 58,000 rps
peak (×3)                      ≈ 175,000 rps

// the ratio that shapes everything
read : write  ≈  100 : 1

// fan-out amplification — the real number
avg followers ≈ 200
5,800 tweets/sec × 200 = 1.2 M timeline writes/sec
   and one celebrity tweet alone can be 100 M
```

Storage

Tweets	500 M/day × ~1 KB ≈ **500 GB/day**, ~180 TB/year. Large, but it is *append-only and partitionable by time*
Media	Dominates by an order of magnitude. Object storage plus CDN, never a database
Timelines	250 M users × 800 tweet **ids** × 8 bytes ≈ **1.6 TB** — small enough to hold in memory, which is the point
Social graph	Billions of edges, but each is tiny. Sharded adjacency lists

The insight the numbers give you. Timelines store **tweet ids, not tweets** — which is what makes keeping every user's timeline in memory affordable. Store the objects once and hydrate at read time from a cache. That single decision is worth more than any technology choice on this sheet.

3 · CAP AND PACELC — PER CAPABILITY, NOT PER SYSTEM

the framing that is usually stated wrongly, and what each part of this platform actually chooses

The correction: CAP is not "pick two of three". You do not *choose* partition tolerance — partitions happen whether you want them or not, so any distributed system must tolerate them. The real statement is narrower: **during a partition**, choose consistency or availability. With no partition you get both. A diagram that says "pick two" and then adds "we always need P" is contradicting itself.

PACELC is the version you actually use. If **P**artition → choose **A** or **C**; **E**lse → choose **L**atency or **C**onsistency. Partitions are rare; the latency-versus- consistency trade-off is made on every single request, which makes it the more useful lens.

And it is chosen per capability. This platform is CP for authentication and AP for timelines — often within one request. "Our system is AP" is the answer of someone who has read about CAP rather than applied it.

Capability	Choice	Why, and what it costs
Authentication & sessions	CP	A revoked session must be revoked. Stale credentials are a security failure, not a stale read
Follow / unfollow	CP	A block or unfollow that does not take effect is a safety issue. Small volume, so strong consistency is affordable
Tweet durability	CP	An acknowledged tweet must exist. Durability before the response returns
Home timeline	AP	Seconds of lag is imperceptible; an unavailable timeline is the product being down
Counts likes, retweets, followers	AP	Approximate and eventually correct is fine — and at this scale exact counters are a distributed-counter problem nobody needs
Search index	AP	Near-real-time. Serving slightly stale results beats an error
Trends	AP	Approximate by construction — see panel 10
Your own tweet	Special	**Read-your-writes is required.** A user must see their own tweet immediately, even while fan-out is still running. Usually solved by injecting it into their own view directly

The last row is the one worth volunteering. Users forgive a friend's tweet arriving late; they do not forgive their own tweet appearing to vanish. Handling read-your-writes explicitly — rather than hoping the fan-out is fast — is a small detail that signals you have thought about the product, not only the architecture.

The trade this product deliberately makes

A tweet appearing in a follower's timeline **seconds late** is invisible to users. A timeline that fails to load is not. That asymmetry is why nearly everything on the read path chooses availability — and it is a *product* judgement expressed as an architectural one.

4 · REFERENCE ARCHITECTURE

write path on the left, read path on the right — and they meet only at the timeline store

EDGE — absorbs the majority of read traffic before any service is involved

Clients (web · mobile · API) → **CDN + WAF** (media, static, DDoS) → **API gateway** (authN · rate limit · route)

Media is served entirely from the CDN and never touches a service. Timelines are personalised and therefore not CDN-cacheable — which is why the read path below exists and why it must be fast on its own.

Rate limiting sits here, after authentication — limiting by identity requires knowing the identity first. Read and write limits differ by an order of magnitude, matching the traffic they protect.

WRITE PATH — small in volume, and the source of all the amplification

1 · Tweet service — validate, assign id (Snowflake — panel 6), idempotency key → **2 · Persist + outbox** — tweet row and event in ONE transaction, **no dual write** → **3 · Respond** — tweet id returned, inject into author's own view immediately → **4 · Kafka** — tweet-created topic, partitioned by author, everything below is async

THE ACKNOWLEDGEMENT BOUNDARY — the single most important line on this diagram
The user is told "posted" after step 3 — once the tweet is **durable**. Fan-out has not started, and must not be waited on.
Making the response wait for fan-out would mean a celebrity's post takes minutes to acknowledge and couples write availability to the **availability of every downstream consumer. Durability is synchronous; distribution is not.**

Fan-out service — the hybrid decision — panel 5 — the core
Search indexer — inverted index — near-real-time
Trend aggregator — sketches over windows — approximate
Notification service — mentions, replies, likes — its own fan-out problem
Media pipeline — transcode, variants — object store + CDN
Counters — likes, retweets, replies — approximate, batched
Why one event, many consumers — Each consumer scales, fails and is replayed independently. None can block the write.

WHY THE OUTBOX, SPECIFICALLY
You cannot atomically write to a database and publish to Kafka. Persist the tweet and its event in one local transaction; a relay publishes from the outbox.
Without it, a crash between the two leaves a tweet that exists but never reaches a single follower's timeline — and nothing errors, so nobody notices.
Consumers are idempotent, because Kafka delivery is at-least-once and a replayed event must not duplicate a timeline entry or double-count a like.

READ PATH — 100× the volume, and it must be a lookup rather than a computation

1 · Timeline service — read cached id list, Redis, ~800 ids, a list read, nothing more → **2 · Pull celebrities** — fetch recent tweets from followed large accounts, **the hybrid half** → **3 · Merge** — combine both sources, dedupe, sort by id, ids are time-ordered → **4 · Rank** — score by relevance, engagement, recency, panel 7 → **5 · Hydrate** — ids → tweet objects from cache, batched, apply deletes and blocks → **6 · Respond** — page of tweets + cursor, p99 target < 200 ms

WHY HYDRATION IS SEPARATE
Timelines hold ids; tweet bodies are stored once and shared. A tweet retweeted a million times is one object, referenced a million times — **and an edit or delete then applies everywhere at once, for free.**

FILTERING HAPPENS AT READ TIME, NOT WRITE TIME
Blocks, mutes, deleted tweets and suspended accounts are applied when the timeline is **read**. Applying them at fan-out would mean rewriting millions of stored timelines every time one user blocks another — which is unbounded work for a trivial action.
The cost is that every read carries a filtering step, so those lists must be small, cached, and cheap to check.

CACHE HIT RATE IS THE WHOLE READ PATH
Timeline lists and tweet objects both live in memory. At this read volume the databases behind them are sized for a **fraction** of real traffic — they physically cannot serve it. A cold cache after a restart is therefore not a slow **period; it is an outage. Warm before taking traffic, and never flush the whole cache at once.**

WHAT MAKES THIS READ FAST — three properties, in order of importance
1 · **It is precomputed.** The expensive merge happened at write time for most followees, so the read is a list lookup rather than a query across hundreds of users.
2 · **It holds ids, not objects.** The list is tiny and stays in memory; bodies are fetched in one batched call against a shared cache.
3 · **Nothing on it is synchronous with anything expensive.** Ranking is bounded, filtering reads small cached sets, and no step waits on a database under normal operation.

STORES — each chosen for one access pattern, and named by what it must do rather than by product

Tweet store — append-only, by id and author, partitioned by time
Timeline cache — Redis list of ids per user, capped, LPUSH + LTRIM
Tweet object cache — id → tweet, shared by all, the hydration source
Social graph — sharded adjacency lists, not a graph database — panel 8
Search index — inverted, time-partitioned, rebuildable projection
User & auth — relational, transactional, small and strict
Object storage — media originals + variants, fronted by CDN
Counters — approximate, batched, never a row per like
Analytics — columnar, clickstream, off the serving path

POLYGLOT PERSISTENCE IS A CONSEQUENCE, NOT A GOAL
Each store above exists because one access pattern genuinely could not be served by the others. Every additional one is a thing to back up, patch, monitor and be paged for — earn each with a measured need.

Naming stores by role rather than by product is deliberate. The interview question is whether you can justify the access pattern, not whether you can recall a vendor. Cassandra, DynamoDB, HBase and Bigtable are all defensible for the tweet store; what matters is that you say *why* a wide-column store with a time-ordered clustering key fits an append-heavy, read-by-author workload.

And note what is deliberately absent: no single database is doing more than one job, and nothing on the read path performs a join. At this scale, a join across services on the request path is the design error that no amount of hardware recovers from.

READING THE DIAGRAM — the three decisions everything else follows from

1 · The write is acknowledged before distribution begins. Durability is synchronous; fan-out is not. This is what keeps post latency constant regardless of whether the author has 200 followers or 100 million, and it is why the event backbone is load-bearing rather than decorative.

2 · The read is a lookup, never a computation. The expensive merge was done in advance for most followees; what remains at read time is a list fetch, a bounded pull for large accounts, a merge, a ranking pass and a batched hydration. Nothing scans, nothing joins.

3 · Consistency is chosen per capability and changes within one request. The same request reads strongly-consistent block lists and eventually-consistent counts, and injects the user's own tweet directly so read-your-writes holds while fan-out is still in flight.

Useful interrogation for any box: what access pattern justifies it, what is its p99, what does the user see when it is down, and what would you remove first if the budget halved?

If a component has no answer to all four, it is on the diagram because it is expected rather than because it is needed — and most published versions of this design carry two or three of those.

5 · THE FAN-OUT PROBLEM — THE HEART OF THIS DESIGN

what the question is actually testing · MID the two strategies · STAFF where the boundary goes and why

The two pure strategies

	Fan-out on write push	Fan-out on read pull
On post	Append the tweet id to every follower's stored timeline	Do nothing
On read	Return the stored list — one lookup	Query every followee's recent tweets, merge and sort
Read latency	**Excellent** — a list read	**Poor** — hundreds of queries per request
Write cost	**O(followers)** — unbounded	**O(1)**
Storage	One entry per follower per tweet	None
Breaks when	Someone has millions of followers	Everyone reads constantly — which they do

Neither works alone, and saying so is the answer. Pure push cannot survive a celebrity: one tweet becomes tens of millions of writes, and if they post ten times a day the fan-out system does nothing else. Pure pull cannot survive the read volume: at ~175,000 reads per second, each fanning out to hundreds of queries, the numbers are absurd.

The hybrid — what is actually built

```
// on post
followers = graph.count(author)

if followers < THRESHOLD:      // ~10k, tuned
    push  → append id to each follower's list
else:
    skip  → store nothing; readers will pull

// on read
timeline = redis.lrange(user, 0, n)        // pushed
celebs   = pullRecent(followedLargeAccounts) // pulled
return rank(merge(timeline, celebs))
```

Why the split works. Ordinary users have few followers, so pushing is cheap. Celebrities have many followers *but post rarely relative to their reach*, so pulling is bounded — and the pull is shared: one fetch of a celebrity's recent tweets serves every follower reading at that moment.

STAFF Choosing the threshold

- **It is a tuning parameter, not a constant.** Order of ten thousand followers is a common landing point, arrived at by measuring — not derived from first principles
- **Too low** → too many accounts pulled, so every read does more work and read latency degrades
- **Too high** → fan-out queues back up during peak posting, and timelines fall behind
- **Consider posting rate as well as follower count.** A million-follower account that posts once a month is cheap to push; a fifty-thousand-follower account posting constantly may be worth pulling
- **It must be changeable at runtime** — a configuration value with a kill switch, not a deploy

The refinement worth mentioning. Fan out only to **active** followers. A large share of accounts have not opened the app in months, and materialising timelines for them is pure waste. Push to recently-active users, and rebuild on demand for anyone returning — this can remove the majority of fan-out work.

What the fan-out worker must handle

- **Idempotency** — Kafka redelivers, and a replayed event must not duplicate the entry
- **Capping** — LPUSH then LTRIM to a fixed length; timelines are bounded, not infinite
- **Batching** — write in chunks rather than one call per follower
- **Backpressure** — lag is the metric that matters; alert on it, not on error rate
- **Prioritisation** — active followers first, so the people who will actually read it get it soonest

STAFF **The question that separates depth: what happens when fan-out lags?** Timelines fall behind silently — no error, no alert unless you built one. Users see stale feeds and nothing in the system considers itself unhealthy. Monitor consumer lag as a primary SLI, shed the lowest value work first, and be able to fall back to pull for accounts whose push is behind.

6 · TWEET IDs — SNOWFLAKE

a detail worth knowing

Every tweet needs a unique id, generated at thousands per second across many machines, with **no coordination** — and ideally **sortable by time**, so that ordering a timeline is ordering integers.

Why the obvious options fail

Auto-increment	Requires a single writer. A coordination point in the hottest path
Random UUID	Unique and coordination-free, but **not sortable** — so ordering needs a separate timestamp and index, and insert locality is poor
Timestamp alone	Collides — many tweets share a millisecond

The Snowflake layout

```
64 bits, most significant first

 1  bit  unused     keeps the value positive
41  bits timestamp  ms since a custom epoch
                    = 69 years of range
10  bits machine    up to 1,024 workers
12  bits sequence   4,096 ids per ms per machine

// capacity: 1,024 × 4,096 = ~4.2 M ids per ms
// far beyond any plausible tweet rate
```

The property that makes it valuable. Because the timestamp occupies the high bits, **sorting ids sorts by time.** Timeline merges, pagination cursors and range scans all become integer comparisons — no separate timestamp column, no secondary index, no clock comparison across machines.

The failure mode to name: clock skew. If a machine's clock moves backwards — an NTP correction, a leap second — it can generate an id it has already issued. The generator must detect a backwards clock and **refuse to issue ids** until time catches up, rather than producing a duplicate. Refusing briefly is recoverable; a duplicate tweet id is not.

The general lesson beyond this system. Sortable, coordination-free identifiers are useful anywhere you need ordering without a central sequence — which is why ULID and UUIDv7 exist and follow the same time-high-bits idea.

7 · TIMELINE STORAGE, RANKING AND THE OPERATIONS THAT ARE HARD AFTERWARDS

MID the cache structure · STAFF ranking, deletion, editing and ordering guarantees

The stored timeline

```
// one capped list of ids per user
key:   timeline:{user_id}
type:  list of tweet ids (8 bytes each)
size:  ~800 entries, hard capped

LPUSH timeline:{u} {tweet_id}
LTRIM timeline:{u} 0 799        // bounded

// 250M users × 800 × 8B ≈ 1.6 TB — fits in memory
// storing tweet OBJECTS instead would be ~100× more
```

Bounding the list is what makes this affordable. Nobody scrolls back a thousand entries in a home timeline; anyone who does can be served from the user timelines directly. Unbounded per-user storage would grow without limit for no product benefit.

Ranking — the timeline is not chronological

- **Candidate generation** — the merged push and pull sets, typically hundreds of tweets
- **Scoring** — engagement signals, author affinity, recency decay, media type, predicted interaction
- **Policy** — deduplicate, cap consecutive tweets from one author, inject what must be shown, remove what must not
- **Cost note** — scoring is bounded because the candidate set is small. The precomputation is what makes ranking affordable at read time

Offer chronological as an option. Some users strongly prefer it, and it is nearly free — it is the merged list before scoring. Being able to say why it costs almost nothing shows you understand where the expense actually sits.

STAFF The operations that are awkward after fan-out

Operation	Why it is hard, and how it is handled
Delete a tweet	The id sits in millions of stored timelines. **Do not remove it from each one.** Delete the object and filter at hydration — a missing or tombstoned object simply does not render. Timelines self-clean as they roll over
Edit a tweet	Free, because timelines hold ids. Update the object once and every timeline shows the new version. Keep an edit history for auditability
Block or mute	Applied at read, from a small cached set. Rewriting stored timelines on every block would be unbounded work for a trivial action
Account suspension	Same mechanism as delete, at the author level — filter at hydration rather than purge
New follow	The new followee's past tweets are absent from the stored timeline. Backfill a bounded window, or let it fill naturally and pull recent ones
Unfollow	Their ids remain until they age out. Filtered at read; not worth rewriting

The principle running through every row: never rewrite fanned-out data. Anything that would require touching millions of stored timelines to satisfy one user action must instead be handled by filtering at read time. This is the single most useful heuristic in this design, and it generalises well beyond it.

STAFF **The ordering guarantee, stated honestly.** There is **no global ordering** across the timeline. Push and pull arrive on different paths with different latencies, so two users may briefly see the same tweets in a different order. Snowflake ids give a consistent sort *once everything has arrived* — the guarantee is eventual convergence, not real-time global order, and claiming otherwise would be false.

8 · THE SOCIAL GRAPH

a correction worth making

A graph database is usually the wrong answer here, and it is a common reflex. Graph engines earn their cost on **variable-depth traversal** — friends-of-friends-of-friends, shortest path, community detection. The follower graph's hot operations are far simpler than that.

What the access patterns actually are

Who follows X?	Read one adjacency list — for fan-out
Who does X follow?	Read one adjacency list — for pull and backfill
Does X follow Y?	A single membership check
How many followers?	An approximate counter

Every one of those is a key lookup or a membership test — which a sharded key-value or wide-column store serves faster, cheaper and more predictably than a graph engine. Store both directions explicitly: followers-of and following-of as separate lists, written together, because both are read hot.

Practical concerns

- **Shard by user id**, so one user's list is on one node
- **Very large lists need paging** — a hundred-million-follower list is never returned whole; cursor over it
- **Counts are approximate** and maintained separately. An exact follower count at this scale is a distributed-counter problem with no product benefit
- **Writes are small but must be strongly consistent** — an unfollow or block that does not take effect is a safety issue, not a stale read
- **Hot shards** — celebrity accounts concentrate load on one partition; cache their lists aggressively

When a graph store *would* be right: a "people you may know" feature doing multi-hop traversal, or fraud-ring detection over the follow graph. Those are genuine graph problems — and they belong in an offline system, not on the serving path.

9 · SEARCH

real-time, over an append-only corpus

Search here has an unusual shape: the corpus is **enormous, append-only and dominated by recency**. Almost all queries want recent tweets; very few want something from years ago. That skew is what the design exploits.

The indexing pipeline

```
tweet → outbox → Kafka → indexer → inverted index
                                 ↳ vector index (semantic)

// never dual-write to the store and the index —
// that leaves the index silently wrong, with no error
```

Partitioning by time

- **Recent shards are small, hot and in memory**; older shards are large, cold and rarely touched
- **Most queries hit only the recent shards**, so the effective search space is a fraction of the corpus
- **Old shards can be moved to cheaper storage** or dropped entirely, according to a retention policy

This is the highest-leverage decision in the panel. Time-partitioning turns "search billions of documents" into "search the last few hours, and only widen if the query demands it".

Query concerns

- **Relevance scoring** — BM25 as a baseline, plus engagement and author signals
- **Recency versus relevance** is a product decision, exposed as top-versus-latest
- **Filters** — author, language, media, date range, safety
- **Hybrid retrieval** — keyword for exact handles and hashtags, vector for meaning. Fuse the ranked lists

The index is a projection, never the source of truth. It must be fully rebuildable from the tweet store, and rebuilding should be a rehearsed operation — you will need it after a mapping change, and discovering that during an incident is the wrong time.

10 · TRENDING TOPICS

approximate on purpose

Trends answer "what is being discussed unusually much, **right now**". Exact counting across billions of terms is neither achievable nor necessary — nobody can tell whether a hashtag was mentioned 48,000 or 49,000 times.

Why exact counting is the wrong goal

- Every tweet contains many candidate terms, so the counting rate is a large multiple of the tweet rate
- The term space is effectively unbounded and constantly changing
- Only the **heavy hitters** matter — the top tens per region, not the full distribution

The approach

Sliding windows	Count over recent intervals — five minutes, an hour — not all time
Probabilistic counting	**Count-Min Sketch** for approximate frequency in fixed memory; heavy-hitter algorithms to surface the top terms
Rate of change	The signal is **acceleration, not volume**. Common words are always frequent; a trend is a term unusually frequent *compared to its own baseline*
Geo and language partitioning	Trends are computed per region, which also bounds the term space per shard
Safety filtering	Applied before publication, not after

The insight to state. Comparing against a baseline is what distinguishes a trend from a popular word. Ranking by raw count surfaces the same generic terms every hour and is useless as a product — which is why the metric is deviation from expectation rather than volume.

Serving is trivial once computed. The output is a small ranked list per region, refreshed every few minutes and cached at the edge. All the engineering is in the aggregation, none in the read.

11 · MEDIA & NOTIFICATIONS

bytes and second-order fan-out

Media — never on the tweet path

1. Client requests a **pre-signed upload URL** and uploads **directly to object storage** — the bytes never pass through your services
2. The upload emits an event; a pipeline transcodes and generates size and format variants
3. The tweet references a media id; the CDN serves the variants
4. The tweet can be posted while processing continues, with a placeholder until ready

Direct-to-storage upload is the decision that matters. Routing media through application servers wastes bandwidth, memory and connection capacity on the busiest tier for no benefit — and it makes a large upload capable of degrading unrelated requests.

Media dominates cost, not tweets. Text is bytes; video is orders of magnitude more, and egress is usually the largest line on the bill. CDN offload ratio is the single highest-leverage cost metric in the whole system.

Notifications — the second fan-out

- **Likes and retweets on a popular tweet** are their own fan-out problem, in the opposite direction — millions of events converging on one author
- **Aggregate rather than deliver individually** — "and 12,000 others" is both better product and vastly cheaper
- **Coalesce over a window** so one person's burst of activity is one notification
- **Mentions of a large account** need rate limiting, or a pile-on becomes a denial of service on the notification path
- **Push delivery is best-effort**; the in-app list is the durable record

A good interview observation: notifications have the *inverse* shape of timelines — many writers, one reader. It is the same fan-out mathematics reflected, and aggregation is the equivalent of the hybrid strategy.

12 · API SPECIFICATION

the contract, including the parts that make it correct under retries and at depth

Method	Endpoint	Notes & status codes
POST	`/v1/tweets`	**201** · **`Idempotency-Key` required** — a retried post must not duplicate · 422 too long · 429
GET	`/v1/tweets/{id}`	**200** · 404 if deleted · 403 if blocked or protected
DELETE	`/v1/tweets/{id}`	**204** · idempotent by nature · 403 if not the author
GET	`/v1/timelines/home`	**200** · **cursor** pagination · the hot path
GET	`/v1/timelines/user/{id}`	**200** · cursor · cacheable, unlike home
POST	`/v1/follows`	**201** · body carries the target · idempotent
DELETE	`/v1/follows/{id}`	**204** — **not** `POST /unfollow`; unfollow is the deletion of a follow
GET	`/v1/users/{id}/followers`	**200** · cursor · **never returns the full list**
POST	`/v1/tweets/{id}/likes`	**201** · idempotent — liking twice is one like
DELETE	`/v1/tweets/{id}/likes`	**204** · unlike
GET	`/v1/search`	**200** · `q, sort=top|latest, filters, cursor`
GET	`/v1/trends`	**200** · by region · heavily cached, short TTL
POST	`/v1/media/upload-url`	**201** · returns a pre-signed URL; bytes go direct to storage
GET	`/v1/notifications`	**200** · cursor · aggregated, not raw events

Post a tweet

```
POST /v1/tweets
Authorization: Bearer …
Idempotency-Key: 01J8F3K2M9Q
{
  "text": "Hello world",
  "media_ids": ["m_1029"],
  "in_reply_to": null
}

HTTP/1.1 201 Created
{
  "id": "1729384756102938624",   // Snowflake
  "author_id": "u_8813",
  "text": "Hello world",
  "created_at": "2026-09-01T10:14:22Z",
  "metrics": { "likes": 0, "retweets": 0,
               "replies": 0 }
}
```

The idempotency key is not optional here. A user on a poor connection taps post, the response is lost, the client retries — and without a key they have tweeted twice publicly. It is the most visible possible failure, and it is entirely preventable.

The id is returned as a string. A 64-bit Snowflake id exceeds what JavaScript represents exactly as a number, so serialising it as an integer silently corrupts the low digits in browser clients. This is a real and well-known class of bug.

Read the home timeline

```
GET /v1/timelines/home?limit=50&cursor=1729384…

HTTP/1.1 200 OK
RateLimit-Limit: 900
RateLimit-Remaining: 887
{
  "data": [ { "id": "172938…", "text": "…",
              "author": { "id": "u_44", "handle": "…" },
              "metrics": { … } }, … ],
  "meta": {
    "next_cursor": "1729384750000000000",
    "result_count": 50
  }
}
```

Cursor pagination, never offset. The timeline changes constantly, so `offset=100` returns a shifting window — users see duplicates and miss tweets. A cursor is a position in an ordered sequence, and because Snowflake ids sort by time, **the id itself is the cursor.**

Conventions applied throughout

- **Versioned in the path**; breaking changes get a new version
- **One error envelope** with a stable machine-readable code
- **Rate limit headers on every response**, not only on 429 — clients can then slow down before being limited
- **Read and write limits differ by an order of magnitude**, matching the traffic each protects

13 · TECHNOLOGY CHOICES — ADVANTAGES, DISADVANTAGES AND THE ALTERNATIVE

every row includes what you would use instead, because "why not X?" is always the second question

Role	Choice	Why	Advantages	Disadvantages	Alternative, and when
Tweet store	Wide-column Cassandra, Bigtable, DynamoDB	Append-heavy, read by author or id, time-ordered, no joins — exactly what a partition key plus clustering key serves	Very high write throughput, linear scaling, no single leader, multi-region friendly	Eventually consistent; no ad-hoc queries; the data model must be designed per access pattern	**PostgreSQL partitioned by time** is entirely adequate below a few hundred million rows — and far simpler
Timeline & object cache	Redis	Capped lists are a native operation, and sub-millisecond reads are what the whole read path depends on	List operations fit the access pattern precisely; extremely low latency; rich types	Memory cost dominates; AP with async replication, so it can lose acknowledged writes on failover	None sensible for the timeline itself. It is a **cache**, rebuildable from source — never the record of truth
Social graph	Sharded key-value or wide-column	The hot operations are list reads and membership tests, not traversal	Predictable, cheap, shards cleanly by user	Multi-hop queries are awkward — but they are not on the serving path	**Graph database** only for genuine traversal features, and run offline rather than in the request path
Event backbone	Kafka	One tweet event feeds fan-out, search, trends, notifications and analytics independently, with replay	High throughput, durable, ordered per partition, replayable, many independent consumers	Operationally heavy; ordering only per partition; consumer lag becomes a primary SLI	**A managed queue** if you only need work distribution. Replay and multiple consumers are what justify a log
Search	Inverted index Elasticsearch, OpenSearch	Relevance ranking, facets and fuzzy matching are a different problem from querying rows	Mature, scales horizontally, strong aggregation, near-real-time indexing	Heavy to run; a second copy of the corpus; must never be authoritative	**PostgreSQL full-text** at small scale — it removes an entire system
User & auth	PostgreSQL	Small, strict, transactional, and it must be correct rather than fast	ACID, constraints, isolation levels, well understood	Single-writer scaling — irrelevant at this volume	None needed. Resist the urge to make everything distributed
Media	Object storage + CDN	Large, immutable, cacheable at the edge, and never touched by application logic	Effectively unlimited, durable, cheap per byte, integrates with the CDN	Egress cost is significant; lifecycle policies are required	None sensible. Serving media from application servers is a mistake at any scale
Analytics	Columnar ClickHouse, BigQuery	Aggregating clickstream and engagement over enormous row counts	Very fast scans and compression; SQL	Poor at point updates; batch-oriented	**Read replicas** until the scan volume genuinely justifies a separate system

The honest close. Every row above is justified by scale that most systems never reach. At a hundredth of this traffic, PostgreSQL with read replicas, Redis and a CDN would serve the entire product, and the fan-out could be a background job — with a fraction of the operational surface. Being able to say where each component *starts* to be necessary is a stronger answer than naming all of them.

14 · PRODUCTION FAILURES, DEGRADATION AND WHAT BREAKS FIRST

the questions asked after the happy path — and the answers that show you have operated something like this

Failure	What you see	Answer
Fan-out lag	Timelines silently stale; no errors anywhere	Alert on consumer lag as a primary SLI; prioritise active users; fall back to pull
Celebrity storm	A very large account posts; fan-out queues saturate	The pull path already covers it — verify the threshold is enforced and tunable at runtime
Cache cold start	Databases receive traffic they were never sized for	Warm before readiness; never flush globally; stagger restarts
Hot key	One trending tweet saturates a single cache node	Replicate the key across nodes, or cache it in-process at the service tier
Duplicate tweet	A user posts twice from one tap, publicly	Idempotency key end to end; the most visible failure on this sheet
Deleted tweet still visible	Content reappears after deletion	Filter at hydration; never rely on purging fanned-out lists

Degradation matrix — decide before the incident

Down	Behaviour
Ranking service	Serve the merged timeline in reverse-chronological order. Slightly worse, entirely usable
Search	Return an honest error for search only; the rest of the product is unaffected
Trends	Serve the last computed list, or hide the module
Notifications	Queue and drain later. Nothing customer-blocking
Fan-out	Reads still work from existing timelines; new tweets arrive late. **Degrade, do not fail**
Media pipeline	Post the tweet; show a placeholder until processing completes
Tweet store	Writes fail loudly and honestly. **Never accept a tweet you cannot durably store**

The principle. Reads degrade gracefully; writes fail loudly. A stale or unranked timeline is acceptable; silently losing a user's post is not.

What breaks first

#	Limit
1	**Fan-out consumer throughput** during a posting peak — it lags before anything errors
2	**Cache memory and hot keys**, particularly around a trending event
3	**Kafka partition skew** — a hot author concentrates load on one partition
4	**Database connection pools**, which saturate long before CPU does
5	**Media egress and cost**, which is a budget limit rather than a technical one

Note that the first three are all fan-out or cache. The tweet database — the thing most designs spend their time on — is nowhere near the top of this list.

What to monitor above all

- **Fan-out consumer lag**, and the age of the oldest unprocessed event
- **Timeline cache hit rate** — a fall here precedes every read incident
- **Timeline read p99**, split by push-heavy and pull-heavy users
- **Post success rate** — the one metric that means the product works
- **Tweets fanned out per second** versus tweets posted per second — the amplification factor, live

15 · INTERVIEW & READINESS CHECKLIST

what a strong answer covers, and what a running system needs

- ☐ **Scope stated**, with what is explicitly out
- ☐ **Read:write ratio derived**, not asserted
- ☐ **Follower distribution raised early** — it is the whole problem
- ☐ **Fan-out amplification calculated**, not hand-waved
- ☐ **Latency targets at p99**, per endpoint
- ☐ **Composite availability computed** for the read chain
- ☐ **CAP stated correctly** — not "pick two"; PACELC per capability
- ☐ **Read-your-writes handled** for the author's own tweet
- ☐ **Hybrid fan-out explained**, with why neither pure strategy works
- ☐ **Threshold is runtime-tunable**, with a kill switch
- ☐ **Fan-out only to active users** considered
- ☐ **Timelines store ids, not objects**, and are capped
- ☐ **Fan-out workers idempotent** — replay must not duplicate
- ☐ **Consumer lag alerted on** as a primary SLI
- ☐ **Outbox used**; no dual write to database and broker
- ☐ **Write acknowledged before fan-out**, never after
- ☐ **Snowflake-style ids** — time-sortable, coordination-free
- ☐ **Clock-skew behaviour defined** — refuse rather than duplicate
- ☐ **Ids serialised as strings** in JSON
- ☐ **Deletes filtered at hydration**, never purged from timelines
- ☐ **Blocks and mutes applied at read**, from cached sets
- ☐ **Edit handled by object update**, with history retained
- ☐ **Ordering guarantee stated honestly** — eventual, not global
- ☐ **Social graph is adjacency lists**, not a graph database
- ☐ **Both graph directions stored** and written together
- ☐ **Follower lists paginated**, never returned whole
- ☐ **Counts approximate** and maintained separately
- ☐ **Search index time-partitioned** and rebuildable
- ☐ **Trends use sketches over windows**, ranked by acceleration
- ☐ **Media uploaded direct to storage** via pre-signed URL
- ☐ **Notifications aggregated**, not delivered per event
- ☐ **Idempotency key on post** — a retry must not tweet twice
- ☐ **Cursor pagination** everywhere; never offset
- ☐ **Rate limits per user and per endpoint**, headers returned on success
- ☐ **Cache warmed before readiness**; no global flush
- ☐ **Degradation matrix agreed**, and each path tested
- ☐ **Reads degrade, writes fail loudly**
- ☐ **Bottleneck order known** — fan-out and cache before the database
- ☐ **Cost per tweet and CDN offload ratio** tracked
- ☐ **Trade-offs to revisit** stated unprompted

The one that summarises the sheet: **trace one tweet from a hundred-million-follower account through to appearing in a specific follower's timeline, and name every place the design deliberately refuses to do the obvious thing.** Acknowledging before fan-out, storing ids rather than objects, pulling instead of pushing, and filtering at read rather than rewriting — each is a refusal, and together they are the design.